

Операционные системы

Лекция 13.2

Свопинг и виртуальная память

План лекции

- Мультипрограммирование со свопингом
- Виртуальная память: основные концепции, страничная, сегментная и сегментно-страничная организации виртуальной памяти
- Задачи управления виртуальной памятью: задача размещения, задача перемещения, задача преобразования адресов, задача замещения
- Подкачка
- Алгоритм замещения страниц

Виртуальное адресное пространство

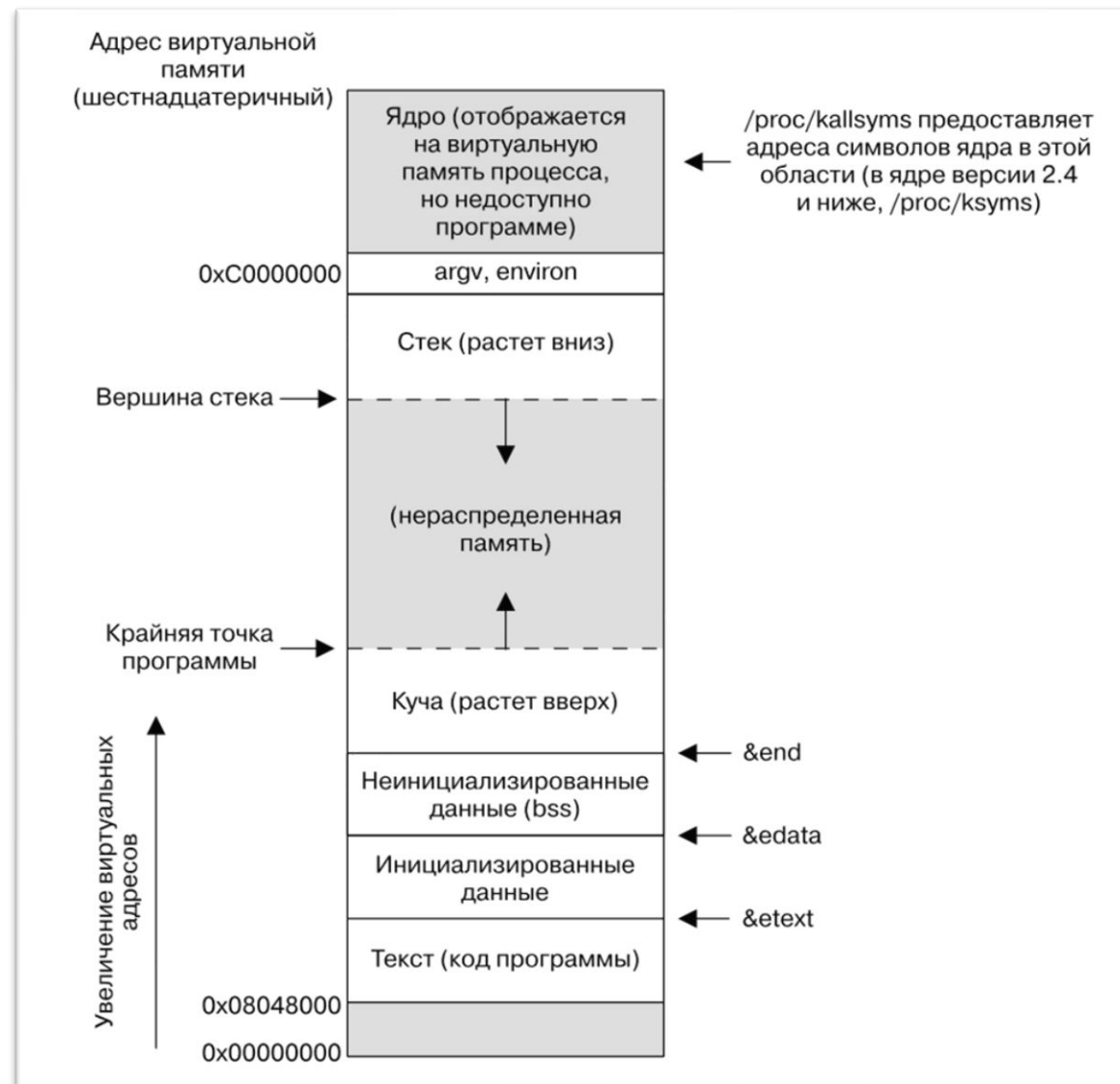
Как уже говорилось **адресное пространство** – это набор адресов, который может быть использован процессом для обращения к памяти

У каждого процесса имеется собственное адресное пространство, независимое от того адресного пространства, которое принадлежит другим процессам

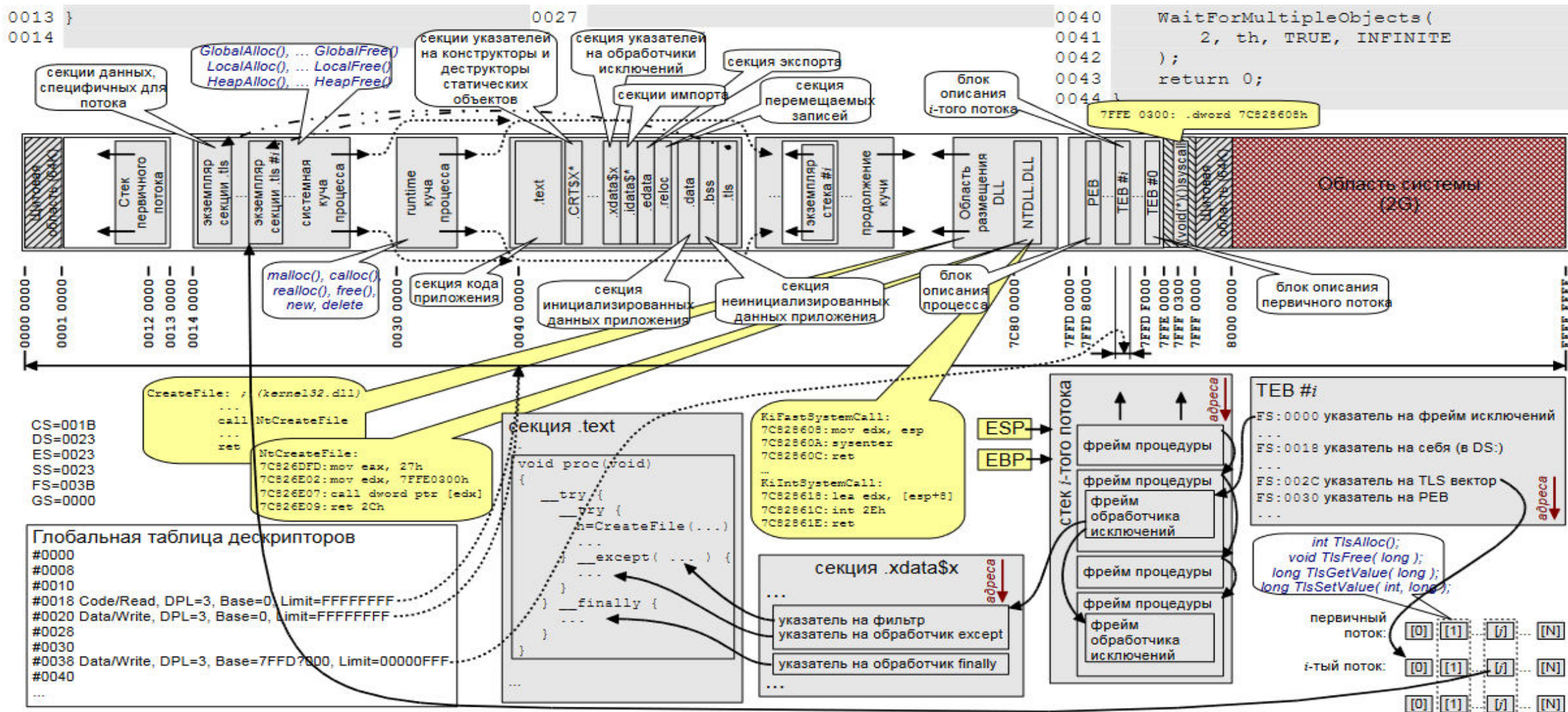
Стоит рассмотреть устройство данного виртуального адресного пространства

Стоит отметить что дальнейшее деление на отдельные области/сегменты является **сугубо логическим** и все области ничем физически между собой не отличаются

Виртуальное адресное пространство



Виртуальное адресное пространство



Виртуальное адресное пространство

Информация о памяти и настройках связанных с памятью в системе:

➤ WinAPI:

- Системная информация – **GetSystemInfo**
- Статус виртуальной памяти – **GlobalMemoryStatusEx**
- Определение состояния адресного пространства – **VirtualQueryEx**, **VirtualQuery**

➤ POSIX:

- Информация об используемых ресурсах – **getrusage**
- Информация об ограничениях на ресурсы – **getrlimit**
- Информация из системных переменных – **sysconf**

Виртуальное адресное пространство

Виды памяти:

- Куча (уровень пользователя)
 - По умолчанию
 - Общая куча, используемая для передачи больших аргументов экземпляру сеанса Csrss.exe процесса (Windows only)
 - Созданная библиотекой времени выполнения языка C
- Стек (уровень пользователя и ядра)
- Свободная виртуальная память (уровень пользователя)
- Пул памяти (уровень ядра)
 - Выгружаемый
 - Невыгружаемый
- Резервные списки (уровень ядра)

Виртуальное адресное пространство

Windows API содержит четыре группы функций для управления памятью в приложениях:

- API Virtual
- API Heap
- Локальные/глобальные API (в целях совместимости с 16-битными приложениями)
- Файлы, отображенные в память

Примечание: работа с памятью в большинстве случаев требует **соблюдения гранулярности** выделяемого объёма памяти. В Windows имеется специальное значение гранулярности, кратное которому должна выделяться память (обычно 64Кб)

Виртуальное адресное пространство

POSIX содержат такие группы функций для управления памятью в приложениях как:

- API mmap
- API heap/stack (аналогично стандартной библиотеке C)
- Файлы, отображенные в память (mmap)

Примечание: В Linux гранулярность памяти основывается на размере виртуальной страницы (обычно 4Кб)

Виртуальное адресное пространство

Действия над виртуальными страницами:

Windows	POSIX
Резервирование региона в адресном пространстве	
Передача памяти зарезервированному региону	
Перевод региона в неактивное состояние и обратно	-
Возврат физической памяти	-
Освобождение региона	
Запрет/разрешение на выгрузку региона из физической памяти	
Смена параметров доступа к региону	

Виртуальное адресное пространство

Действия над виртуальными страницами (API):

Windows	POSIX
<u>VirtualAlloc</u> с MEM_RESERVE	<u>mmap</u> с PROT_NONE
<u>VirtualAlloc</u> с MEM_COMMIT	<u>mprotect</u> со значением отличным от PROT_NONE
<u>VirtualAlloc</u> с MEM_RESET	-
<u>VirtualFree</u> с MEM_RELEASE	-
<u>VirtualFree</u> с MEM_DECOMMIT	<u>munmap</u>
<u>VirtualLock</u> <u>VirtualUnlock</u>	<u>mlock</u> <u>munlock</u>
<u>VirtualProtect</u> (флаги)	<u>mprotect</u>

Виртуальное адресное пространство

Дополнительные функции для работы с памятью:

➤ WinAPI:

- управление рабочим множеством – GetProcessWorkingSetSize, SetProcessWorkingSetSize
- Операции над памятью – FillMemory, CopyMemory, MoveMemory, SecureZeroMemory, ReadProcessMemory, WriteProcessMemory

➤ POSIX:

- Операции над памятью – memcpy, memmove, memccpy, memset

Напомним, что **рабочее множество процесса** – это набор страниц виртуальной памяти, которые процесс активно использует в текущий момент времени

Очевидно, что данный набор является не постоянным как по составу, так и по размеру

Виртуальное адресное пространство

Когда работать с виртуальной памятью напрямую:

- При разработке системного программного обеспечения
- Когда в приложении требуется максимальный контроль над выделением памяти (кучи не могут похвастаться таким)
- Когда планируется большое количество операций с большими массивами объектов или структур
- Когда планируется реализовать свой собственный алгоритм выделения памяти пользовательского уровня

Виртуальное адресное пространство

Куча – это регион зарезервированного адресного пространства. Первоначально большей его части физическая память не передается. По мере того, как программа занимает эту область под данные, специальный диспетчер, управляющий кучами (heap manager), постранично передает ей физическую память (из страничного файла). А при освобождении блоков в куче диспетчер возвращает системе соответствующие страницы физической памяти

Динамическая память, или **куча** – область, из которой память (для переменных) может динамически выделяться в ходе выполнения программы. Верхний конец кучи называют **крайней точкой программы**

Виртуальное адресное пространство

Действия над кучей:

Windows	POSIX
Создание кучи	-
Выделение области памяти в куче	
Перераспределение области памяти в куче	
Очистка области памяти в куче	
Удаление кучи	-
Получение кучи процесса	-
-	Установка крайней точки программы

Виртуальное адресное пространство

Действия над кучей (API):

Windows	POSIX + Windows
<u>HeapCreate</u>	-
<u>HeapAlloc</u>	<u>malloc</u> , <u>calloc</u>
<u>HeapReAlloc</u>	<u>realloc</u>
<u>HeapFree</u>	<u>free</u>
<u>HeapDestroy</u>	-
<u>GetProcessHeap</u>	-
-	<u>brk</u> , <u>sbrk</u> (Non-windows)

Виртуальное адресное пространство

Дополнительные функции для работы с кучей в WinAPI:

- **HeapLock** – занятие кучи потоком (для синхронизации доступа к куче)
- **HeapUnlock** – освобождение кучи потоком
- **HeapCompact** – уплотнение кучи (для борьбы с фрагментацией)
- **HeapValidate** – проверка целостности кучи
- **HeapWalk** – позволяет просматривать содержимое кучи (для отладки)

Виртуальное адресное пространство

Когда стоит использовать кучи:

- Наиболее подходящие для работы с множеством малых объектов
- Когда в приложении не требуется полный контроль над выделением памяти
- Когда требуется точное выделение памяти (память в кучах не подвержена гранулярности, но при создании самих куч правило гранулярности работает)

Виртуальное адресное пространство

Когда стоит создавать отдельные кучи (WinAPI):

- Защита компонентов
- Более эффективное управление памятью (по сравнению с использованием общей кучи)
- Локальный доступ
- Исключение издержек, связанных с синхронизацией потоков
- Быстрое освобождение всей памяти в куче

Виртуальное адресное пространство

Программный поток должен иметь доступ к временной области памяти для хранения параметров функций, локальных переменных и адреса возврата после вызова функции. Эта часть памяти называется **стеком (stack)**

Стек является довольно статичной частью адресного пространства, в том смысле, что практически не существует инструментов взаимодействия с ним на пользовательском уровне

Примечание: существует функция **alloca** для выделения памяти в стеке. Однако её использование является **крайне не рекомендованным!**

Виртуальное адресное пространство

Советы по работе со стеком:

- Избегайте использования стека для динамических структур. Используйте кучу, а в стек записывайте указатель на структуру в куче
- Внимательно работайте с большими локальными переменными. Такие переменные быстро заполнят стек и приведут к ошибке переполнения стека
- Избегайте рекурсий с большими структурами данными (по тем же причинам, что и в прошлом пункте)
- Если требуется увеличить размер стека, делайте это рассудительно, так как выделение большего количества памяти, чем требуется может привести к неоправданному потреблению памяти и ошибкам «Out-of-memory»

Виртуальное адресное пространство

На стадии инициализации системы диспетчер памяти создает два пула памяти, или кучи (heap), с динамически изменяемым размером; они используются большинством компонентов режима ядра для выделения системной памяти:

- **Невыгружаемый пул** – состоит из диапазонов системных виртуальных адресов, которые заведомо будут постоянно находиться в физической памяти. Таким образом, к этим адресам можно обратиться в любой момент без возникновения ошибки страницы
- **Выгружаемый пул** – область виртуальной памяти в системном пространстве, содержимое которой может подгружаться в систему и выгружаться из нее

Виртуальное адресное пространство

Тегирование пулов – это механизм, который позволяет присвоить уникальный идентификатор, называемый "тегом", пулу памяти или ресурсов ввода/вывода. Этот тег можно использовать для отслеживания и управления ресурсами, связанными с конкретным процессом, потоком или компонентом

Тегирование пулов может помочь оптимизировать производительность системы, позволяя определить и управлять ресурсами, которые вызывают задержки или проблемы с производительностью

Тегирование пулов может помочь в устранении неполадок, предоставляя способ определить источник утечек ресурсов или других проблем

Виртуальное адресное пространство

Windows также предоставляет быстрый механизм выделения памяти – так называемые **резервные списки (look-aside lists)**. Основное различие между пулами и резервными списками заключается в том, что из пула могут выделяться блоки памяти переменного размера, тогда как резервные списки содержат только блоки фиксированного размера. И хотя обобщенные пулы памяти более гибки в отношении того, что они могут поддерживать, резервные списки работают быстрее, потому что они не используют спин-блокировки

Виртуальное адресное пространство

Советы при работе с памятью:

- Когда возможно используйте стандартные функции выделения памяти (malloc, calloc, free, ...), вместо работы с виртуальной памятью напрямую
- Всегда проверяйте возвращаемое значение функций выделения памяти, чтобы убедиться, что выделение прошло успешно
- Всегда освобождайте выделенную память, когда она больше не нужна, чтобы предотвратить утечки памяти
- Используйте память стека (например, локальные переменные) вместо памяти кучи (например, динамически выделенной памяти), когда это возможно
- Избегайте использования глобальных переменных, так как они могут привести к утечкам памяти и сделать код более сложным
- Избегайте использования сырых указателей (например, int*), когда это возможно, так как они могут привести к утечкам памяти и висячим указателям

Операционные системы

Лекция 13.2

Свопинг и виртуальная память